



南開大學
Nankai University

计算机学院

并行程序设计期末研究报告

高斯消去算法的多架构并行化
与 Gröbner 基计算探索 姓名：王毅豪

学号：2412509

专业：计算机科学与技术

2026 年 7 月 4 日

目录

1 问题描述与研究路线	3
1.1 高斯消去法	3
1.2 研究路线	3
1.3 实验平台总览	3
2 多架构统一算法设计	4
2.1 并行维度分析	4
2.2 SIMD: NEON 向量化	4
2.3 Pthread 与 OpenMP 多线程对比	5
2.4 MPI 分布式并行	6
2.5 CUDA GPU 加速	7
3 综合性能数据与对比分析	7
3.1 完整性能数据与强扩展分析	7
3.2 Pthread vs OpenMP 详细对比	8
3.3 MPI 通信复杂度形式化分析	8
3.4 八进程测试的集群资源限制	9
3.5 CUDA GPU 加速的规模分析	9
3.6 CUDA 调试过程: 两个关键 Bug	9
3.7 与 GPU 编程课程 (HIP) 的知识衔接	9
3.8 调试经验跨架构汇总	10
3.9 加速比规律分析	10
3.10 时间分解与瓶颈建模	11
3.11 编程复杂度对比	11
4 【新增内容】Gröbner 基高斯消去	12
4.1 问题背景与区别	12
4.2 串行算法描述	12
4.3 位向量数据结构	12
5 【新增内容】Gröbner 基的并行化实现	13
5.1 NEON SIMD 向量化 XOR	13
5.2 MPI 多进程并行	14
6 【新增内容】Gröbner 基性能测试与分析	15
6.1 测试方案	15
6.2 性能结果	16
6.3 结果分析	17
6.4 调试经验与代码完整性	17
6.5 通信时间占比实测	18
6.6 稀疏矩阵与位图矩阵的选择	18
6.7 NEON SIMD 的向量化前提条件讨论	18

6.8 与 Xsimd 跨平台 SIMD 方案的对比讨论	19
6.9 MPI 通信复杂度形式化分析	19
6.10 调试验证与正确性保证	19
6.11 Gröbner 基算法完整 Walkthrough 示例	20
6.12 MPI 从进程代码详解	20
6.13 位图 vs 稀疏存储的性能权衡	21
6.14 GPU 加速 Gröbner 基的前景与挑战	22
6.15 Gröbner 基部分的总结	22
6.16 普通 vs 特殊高斯消去的并行化对比	22
6.17 MPI 通信实验：并行效率的定量分析	23
6.18 N 规模对 MPI 通信开销的影响	23
6.19 Pthread 多线程规模效应详析	23
7 总结	24

1 问题描述与研究路线

1.1 高斯消去法

高斯消去法（Gaussian Elimination）是求解线性方程组 $Ax = b$ 的经典算法，也是 LU 分解、矩阵求逆等更复杂数值代数问题的基础。给定 $N \times N$ 系数矩阵 A ，算法通过消去过程将其转化为上三角矩阵。标准串行伪代码如下：

普通高斯消去串行算法

```

1 for k = 0 to N-1:
2     for j = k+1 to N-1: A[k][j] = A[k][j] / A[k][k]      // 归一化
3     A[k][k] = 1.0
4     for i = k+1 to N-1:
5         for j = k+1 to N-1: A[i][j] -= A[i][k] * A[k][j]  // 消去
6         A[i][k] = 0.0

```

时间复杂度为 $O(N^3)$ ，其中核心消去操作 $\sum_{k=0}^{N-1} (N-k)^2 \approx N^3/3$ 次浮点乘减。消去步骤的行间无数据依赖，是并行化的主要目标。矩阵初始化采用严格对角占优策略——确保 $|A_{ii}| > \sum_{j \neq i} |A_{ij}|$ ，避免消去过程中出现 inf/nan。

1.2 研究路线

本文的总体研究路线分为三个阶段：

- 单核 SIMD 向量化 (Lab2)：**利用 ARM NEON 128 位向量指令集，在最内层 j 循环实现 4 路单精度浮点并发，探索指令级并行 (ILP) 的加速上限
- 单节点多核并行 (Lab3)：**在鲲鹏 920 (8 核) 上使用 Pthread 和 OpenMP 两种模型，探索共享内存架构下的线程级并行 (TLP) 和同步机制的影响
- 跨节点分布式并行 (Lab4)：**使用 MPI 在鲲鹏集群上实现多进程并行，探索分布式内存架构下的通信开销对加速比的影响
- GPU 众核并行 (Lab5)：**使用 CUDA 在 NVIDIA Tesla T4 上实现，探索 GPU 的 SIMT 模型对计算密集任务的加速效果

前四个阶段使用的所有代码、数据和分析构成了本文基础部分。在此基础上，本文新增 Gröbner 基高斯消去的并行化探索 (§5-§7)，完整代码和数据均为本学期期末阶段的新工作。

1.3 实验平台总览

表 1: 四种实验平台关键参数

	NEON	OpenMP/Pthread	MPI	CUDA
CPU/GPU	鲲鹏 920(1 核)	鲲鹏 920(8 核)	鲲鹏集群	Tesla T4
架构	ARM AArch64	ARM AArch64	ARM $\times 17$ 节点	NVIDIA Turing
并行单元	4 lanes	8 线程	4 进程 $\times$ 8 核	5120 CUDA
内存模型	共享寄存器	共享内存	分布式	独立显存
编译器	GCC -O2	GCC -O2 -fopenmp	mpicxx -O2	nvcc -O2

2 多架构统一算法设计

2.1 并行维度分析

普通高斯消去的三层嵌套循环包含两个可并行的维度：

- **最内层 j 循环：**归一化 ($A[k][j]/A[k][k]$) 和消去 ($A[i][j] - A[i][k] \times A[k][j]$) 均为元素间独立操作，适合 SIMD 向量化——将连续内存元素打包为固定宽度的向量寄存器，单指令处理多数据
- **第二层 i 循环：**各行消去互不影响（行间无依赖），适合多线程/多进程/GPU Block 级并行——将行集划分给不同执行单元独立处理

四种架构在同一个算法框架下，区别仅在于”拆分哪层循环、分配哪种粒度的执行单元、使用什么同步机制”三个设计选择。表 2 总结了各架构的设计决策。

表 2: 四架构并行化设计统一框架

设计维度	NEON	OpenMP	MPI	CUDA
拆分循环	最内层 j	第二层 i	第二层 i	$i(\text{Block})+j(\text{Thread})$
执行单元数	4 lanes	1-8 线程	2-4 进程	5120 cores
内存模型	共享寄存器	共享内存	分布式内存	独立显存
同步方式	指令隐式同步	#pragma 隐式 barrier	MPI_Bcast	cudaDeviceSynchronize
数据划分	不需要	不需要	初始分发 + 逐行广播	cudaMemcpy H2D/D2H
$A[k][k]$ 问题	无需考虑	共享读取	持有者独占	需拆分 Kernel

2.2 SIMD: NEON 向量化

ARM NEON 提供 128 位向量寄存器，可同时容纳 4 个单精度浮点数。核函数将归一化循环和消去循环分别向量化。关键优化包括：

- **除法转乘法：**预计算主元倒数 $1.0/A[k][k]$ ，用 `vmulq_f32` (3 cycles) 替代 `vdivq_f32` (10 cycles)，将除法循环的延迟降低约 3 倍
- **融合乘减 (FMA)：**消去循环使用 `vmlsq_f32` 指令，单周期内同时完成乘法和减法，并行处理 4 路
- **循环展开与尾部处理：**主体按步长 4 推进 ($j+=4$)，末尾不足 4 个元素退化标量串行处理
- **非对齐访存：**AArch64 架构原生支持未对齐内存访问，使用 `vld1q_f32` 直接加载，省去对齐边界判断逻辑

NEON 向量化核心代码

```

1 #include <arm_neon.h>
2 void gauss_neon(float** A, int n) {
3     for (int k = 0; k < n; k++) {
4         float inv = 1.0f / A[k][k];
5         float32x4_t v_inv = vdupq_n_f32(inv);
6         int j = k + 1;
7         // 归一化：向量化乘法
8         for (; j <= n - 4; j += 4) {
9             float32x4_t v = vld1q_f32(&A[k][j]);

```

```

10         v = vmulq_f32(v, v_inv);
11         vst1q_f32(&A[k][j], v);
12     }
13     for (; j < n; j++) A[k][j] *= inv;
14     A[k][k] = 1.0f;
15     // 消去：融合乘减
16     for (int i = k + 1; i < n; i++) {
17         float32x4_t vik = vdupq_n_f32(A[i][k]);
18         for (j = k + 1; j <= n - 4; j += 4) {
19             float32x4_t vkj = vld1q_f32(&A[k][j]);
20             float32x4_t vij = vld1q_f32(&A[i][j]);
21             vij = vmlsq_f32(vij, vkj, vik); // 单周期融合乘减
22             vst1q_f32(&A[i][j], vij);
23         }
24         for (; j < n; j++) A[i][j] -= A[i][k] * A[k][j];
25         A[i][k] = 0.0f;
26     }
27 }
28 }

```

2.3 Pthread 与 OpenMP 多线程对比

两种多线程模型均采用静态线程池范式，关键差异在于同步机制和负载均衡策略。

Pthread 版本采用循环划分 + Barrier 同步：每轮消去中，线程 0 负责归一化后，所有线程通过 `pthread_barrier_wait` 同步，再按循环划分处理各自的行集。N 轮消去共需 $2N$ 次屏障同步（每轮归一化后和消去后各一次），每次涉及内核态切换。当 $N=1024$ 时，2048 次 barrier 切换的累计延迟约 400-1000 ms，严重拖累小规模性能。

OpenMP 版本仅需 3 条编译制导指令：

`pragma omp parallel` 创建线程池，

`pragma omp single` 归一化，

`pragma omp for schedule(dynamic, 16)` 动态调度消去行。隐式 barrier 由 OpenMP 运行时（基于线程池和用户态自旋等待）优化管理，无需内核态切换。动态调度（`chunk_size=16`）有效缓解了消去后期的倒三角负载不均——后期剩余行数减少但每行计算量不同，动态分配让快线程自动承接更多工作。

Pthread Barrier 版本（精简）

```

1 void* pthread_gauss_func(void* arg) {
2     ThreadParam* p = (ThreadParam*)arg;
3     pthread_barrier_t* bar = p->barrier;
4     for (int k = 0; k < N; k++) {
5         if (p->t_id == 0) { /* 归一化 */ A[k][k] = 1.0f; }
6         pthread_barrier_wait(bar); // 同步点 1
7         for (int i = k+1+p->t_id; i < N; i += NUM_THREADS) { // 循环划分
8             float f = A[i][k];
9             for (int j = k+1; j < N; j++) A[i][j] -= f * A[k][j];
10            A[i][k] = 0.0f;
11        }
12        pthread_barrier_wait(bar); // 同步点 2

```

```

13     }
14     return NULL;
15 }

```

OpenMP 版本（精简）

```

1 void gauss_openmp(float** m, int N) {
2     omp_set_num_threads(NUM_THREADS);
3     #pragma omp parallel shared(m, N)           // 线程池
4     for (int k = 0; k < N; k++) {
5         #pragma omp single                       // 单线程归一化
6         { float inv = 1.0f / m[k][k];
7             for (int j = k+1; j < N; j++) m[k][j] *= inv;
8             m[k][k] = 1.0f; }
9         #pragma omp for schedule(dynamic, 16)    // 动态调度消去
10        for (int i = k+1; i < N; i++) {
11            float f = m[i][k];
12            for (int j = k+1; j < N; j++) m[i][j] -= f * m[k][j];
13            m[i][k] = 0.0f;
14        }
15    }
16 }

```

完整的 Pthread vs OpenMP 性能对比数据和 NEON 结合版本见 Lab3 实验报告。核心结论是：OpenMP 在编程复杂度（~15 行 vs ~80 行）、小规模性能（barrier 内核切换 vs 用户态自旋）和负载均衡（dynamic 调度 vs 静态循环划分）三个维度全面优于 Pthread。

2.4 MPI 分布式并行

MPI 版本的核心挑战在于分布式内存架构下的通信设计。采用一维行块划分，数据仅在 0 号进程初始化时分发一次，之后每轮消去仅广播一行（ N 个 float，约 4KB），通信量从常规主从模式的 $O(P \times N^3)$ 降至 $O(N^2)$ 。

MPI 主循环

```

1 double gauss_mpi(float* local_a, int N, int rank, int size) {
2     int base = N/size, rem = N%size;
3     int start = (rank < rem) ? rank*(base+1) : rank*base+rem;
4     int lrows = (rank < rem) ? (base+1) : base;
5     float* krow = alloc_vec(N);
6     for (int k = 0; k < N; k++) {
7         int owner = (k < rem*(base+1)) ? k/(base+1)
8                     : (k - rem*(base+1))/base + rem;
9         if (rank == owner) {
10            int lk = k - start;
11            float inv = 1.0f / local_a[lk*N + k];
12            for (int j = k+1; j < N; j++) local_a[lk*N + j] *= inv;
13            local_a[lk*N + k] = 1.0f;
14            memcpy(krow, &local_a[lk*N], N*sizeof(float));
15        }

```

```

16     MPI_Bcast(krow, N, MPI_FLOAT, owner, MPI_COMM_WORLD);
17     for (int li = 0; li < lrows; li++) {
18         int gi = start+li; if (gi <= k) continue;
19         float f = local_a[li*N+k];
20         for (int j = k+1; j < N; j++) local_a[li*N+j] -= f * krow[j];
21         local_a[li*N+k] = 0.0f;
22     }
23 }
24 return MPI_Wtime() - t0;
25 }

```

段错误调试经验：初版使用 *posix_memalign* + 伪二维指针在跨节点 MPI 环境下出现 segfault。改为一维 *calloc* + 显式偏移 $A[i \times N + j]$ 后解决。

2.5 CUDA GPU 加速

GPU 版本将归一化和消去拆分为两个独立 CUDA Kernel。Division Kernel 采用列并行（每个线程一列），Elimination Kernel 采用行并行（每个 Block 一行，Block 内 256 线程覆盖该行全部列）。

CUDA 双 Kernel

```

1  __global__ void division_kernel(float* A, int k, int N) {
2      int tid = blockIdx.x*blockDim.x + threadIdx.x;
3      int col = k + 1 + tid;
4      if (col < N) { float d = A[k*N+k]; if (fabs(d)>1e-30f) A[k*N+col]/=d; }
5  }
6  __global__ void set_diag_kernel(float* A, int k, int N) { A[k*N+k] = 1.0f; }
7  __global__ void eliminate_kernel(float* A, int k, int N) {
8      int row = k + 1 + blockIdx.x; if (row >= N) return;
9      float f = A[row*N+k];
10     for (int c = k+1+threadIdx.x; c < N; c += blockDim.x)
11         A[row*N+c] -= f * A[k*N+c];
12     __syncthreads(); if (threadIdx.x == 0) A[row*N+k] = 0.0f;
13 }

```

关键设计决策：Division 和 Set_Diag 必须拆为两个 Kernel——Division Kernel 的多 Block 并发读取 $A[k][k]$ 后再由各 Block thread 0 写入 $A[k][k] = 1.0$ 会产生未定义行为（多 Block 间无同步机制）。拆分为两个 Kernel 后，Set_Diag 仅 1 个线程执行，消除竞争。

$N=2048$ 时达到 19.90x 加速比； $N=512$ 时因 $3N$ 次 *cudaDeviceSynchronize* 开销（每次 5-10 μs ）加速比仅为 2.60x。

3 综合性能数据与对比分析

3.1 完整性能数据与强扩展分析

表 3 详细列出了四次实验在所有测试配置下的完整数据。

表 3: 全部实验性能数据汇总

实验	N	配置	串行 (ms)	并行 (ms)	加速比
NEON	512	4 lanes	47.03	22.41	2.10x
	1024	4 lanes	426.94	207.86	2.05x
	2048	4 lanes	3575.58	1730.09	2.07x
OMP+NEON	512	2 线程	48.51	15.41	3.15x
	512	4 线程	48.51	20.58	2.36x
	512	8 线程	48.51	12.88	3.77x
	1024	4 线程	446.13	146.45	3.05x
	1024	8 线程	446.13	86.89	5.13x
	2048	4 线程	4556.36	725.22	6.28x
	2048	8 线程	4556.36	472.89	9.64x
Pthread	512	2 线程	48.51	99.75	0.49x
	1024	2 线程	446.13	958.39	0.47x
	1024	4 线程	446.13	573.84	0.78x
	2048	8 线程	4556.36	1107.23	4.12x
MPI	512	2 进程	55.71	43.43	1.28x
	512	4 进程	51.01	28.29	1.80x
	1024	2 进程	468.37	311.82	1.50x
	1024	4 进程	428.08	188.24	2.27x
	2048	2 进程	3371.45	2550.94	1.32x
	2048	4 进程	3274.10	1345.93	2.43x
CUDA	512	T4 GPU	29.76	11.44	2.60x
	1024	T4 GPU	403.89	26.56	15.21x
	2048	T4 GPU	2832.34	142.34	19.90x

3.2 Pthread vs OpenMP 详细对比

Pthread 和 OpenMP 的本质差异在于同步机制。在 $N=1024$ 、2 线程的极端条件下，Pthread 的性能灾难（0.47x，比串行还慢 2 倍）直接归因于 2048 次 `pthread_barrier_wait` 调用。每次 barrier 调用需要进入内核态（futex 系统调用），延迟约 0.2-0.5 ms，累积 400-1000 ms 的同步开销——而整个计算仅需约 200 ms（串行 446 ms / 2）。这就是多线程小规模场景下的“同步税”（synchronization tax）。

OpenMP 通过运行时优化的用户态自旋等待（user-mode spin-wait），将 barrier 延迟降低到 20-50 μ s 量级——比 Pthread 快约 10 倍。这也是为什么 OpenMP 在 $N=512/2$ 线程下仍有 1.57x 加速比，而 Pthread 仅为 0.49x。

随着 N 增大至 2048，计算时间增长至秒级，同步开销的占比逐步降低。Pthread 在 $N=2048/8$ 线程下勉强达到 4.12x，但与 OpenMP+NEON 的 9.64x 相比仍有 2.3 倍的差距——后者得益于 NEON 向量化（约 2x 贡献）和更优的线程调度（隐式 barrier 和动态负载均衡）。

3.3 MPI 通信复杂度形式化分析

MPI 高斯消去的通信瓶颈可以通过通信-计算比（Communication-to-Computation Ratio, CCR）来精确刻画：

$$\text{CCR} = \frac{N \times T_{\text{bcast}}(4N, P)}{T_{\text{serial}}/P} \quad (1)$$

其中 $T_{\text{bcast}}(M, P) = \alpha \log P + \beta M$ 是二叉树广播算法的时间（ α 为启动延迟，约 0.5 ms； β 为每字节传输时间， $\approx 1/\text{}$ ）。

对于 $N=2048, P=4$ ： $T_{\text{serial}} = 3274 \text{ ms}$ ， $P = 4$ ， $T_{\text{bcast}}(8KB, 4) \approx 0.5 \log 4 + 8 \times 10^3 / 10^9 \approx 1 + 0.008 \approx 1 \text{ ms}$ 。2048 次广播累积延迟 2048 ms，占总时间的 $2048 / (3274/4 + 2048) \approx 71\%$ 。实际加速比 2.43x 对应的 CCR 约 49%，说明其他开销（串行部分、同步、结果收集）也占了可观比例。

3.4 八进程测试的集群资源限制

MPI 实验原计划测试 $P=8$ 的强扩展性，但在实际提交过程中遭遇了集群资源瓶颈。鲲鹏集群共 17 个计算节点，由并行程序设计课程全体同学（约 60 人）共享。PBS 调度系统采用 FCFS（先到先服务）策略，8 个完整节点的同时分配在工作日白天几乎是不可完成的——多次提交后作业始终处于排队（Q）状态，最终放弃该配置的测试。这一经历体现了真实 HPC 环境中资源共享和调度策略对实验设计的影响。

3.5 CUDA GPU 加速的规模分析

NVIDIA Tesla T4 GPU 上的 CUDA 高斯消去展现了显著的规模效应：

- **N=512:** 仅 2.60x 加速。矩阵太小（1 MB）， $3N = 1536$ 次 *cudaDeviceSynchronize* 的累积开销（每次 5-10 μs ，总计约 15 ms）在总 GPU 时间（11.44 ms）中估计占约 13 ms——实际 Kernel 计算时间几乎为零。GPU 在此规模下完全被启动开销拖垮。
- **N=1024:** 15.21x 加速。Kernel 启动开销（约 30 ms）相对总时间（26.56 ms）的占比降至约 50%，GPU 计算开始发挥作用。但在 Tesla T4 的 5120 CUDA Cores 面前， $1024^3 \approx 10^9$ FLOPs 的计算量仍显不足。
- **N=2048:** 19.90x 加速。计算量 $O(N^3)$ 增长至约 8×10^9 FLOPs，Kernel 启动开销（约 61 ms）占总 GPU 时间（142 ms）的比例降至 43%。GPU 的规模优势开始充分释放。随着 N 继续增大，预计加速比将持续攀升——这也是 GPU 适合“大家伙”而非“小问题”的典型特征。

3.6 CUDA 调试过程：两个关键 Bug

Bug 1 — 对角线为垃圾值（12,907 而非 1.0）：初版在 Division Kernel 中由每 Block 的 thread 0 设置 $A[k][k] = 1.0$ ，多 Block 并发写入同一内存位置产生 race condition，导致某 Block 在读取 $A[k][k]$ 之前另一个 Block 已将其修改为 1.0。修复方案：将 Division 和 Set_Diag 拆分为两个独立 Kernel——Set_Diag 仅启动 1 线程（ $\lll 1, 1 \ggg$ ），消除并发写入。

Bug 2 — N=2048 产生 inf：初版使用上三角 + 行组合的矩阵初始化（同 Lab2/Lab3），在 $N=2048$ 时由于行累加导致对角元素值溢出 float 表示范围（约 10^{38} ）。修复方案：改用严格对角占优矩阵（ $|A_{ii}| > \sum_{j \neq i} |A_{ij}|$ ），确保对角元素始终有界且在消去过程中不会产生零主元。

3.7 与 GPU 编程课程（HIP）的知识衔接

在 Lab5 基础部分的 AMD HIP 编程课程中，六个实验涵盖了 GPU 编程的核心概念。这些概念与 CUDA 高斯消去的设计形成直接对应：Vector Addition 的线程索引 $idx = blockIdx.x \times blockDim.x + threadIdx.x$ 直接映射到 Division Kernel 的列索引计算；Matrix Transpose 的内存合并访问分析解释了为何 Elimination Kernel 的“每个 Block 一行 + 线程列遍历”映射（grid-stride loop）是高效的选择——连续线程访问同行连续列，满足 coalesced read；Histogram 的两级归约（LDS $\rightarrow$ global）为未来实现消去行数超过 GPU 核心数时的合并策略提供了模板；Reduction 的多元素/线程 grid-stride loop 直接应用于 Elimination Kernel 的列覆盖模式；K-Means 的三 Kernel 链式调用（Assignment $\rightarrow$ Update $\rightarrow$ Converge）与 CUDA 高斯消去的 Division $\rightarrow$ Set_Diag $\rightarrow$ Elimination 链式调用模式完全一致。

3.8 调试经验跨架构汇总

表 4 汇集了本学期五次实验中在各架构上遇到的关键 bug 及解决方案，这些经验本身构成了一组有价值的并行程式除错知识库。

表 4: 跨架构调试经验汇总

架构	现象	根因	解决方案
NEON	除法指令性能低	vdivq_f32 延迟 10 cycles	倒数法 vmulq_f32 3 cycles
Pthread	2 线程慢于串行	2048 次内核态 barrier	换用 OpenMP
MPI	Segfault	posix_memalign 兼容性	calloc + 一维数组
MPI	mpi.sh 损坏	sed 全部替换了数字	手动编辑脚本
MPI	8 进程排队	集群资源竞争	放弃, 报告中说明
CUDA	对角线 =12907	多 Block 并发写竞争	拆分 Division+Set_Diag
CUDA	N=2048 出现 inf	浮点累加溢出	严格对角占优矩阵
Gröb.MPI	P=4 double free	vector RAII 冲突	calloc 替代 vector
Gröb.MPI	P=4 死循环	j++ 在 if 内	移至 MPI_Bcast 后

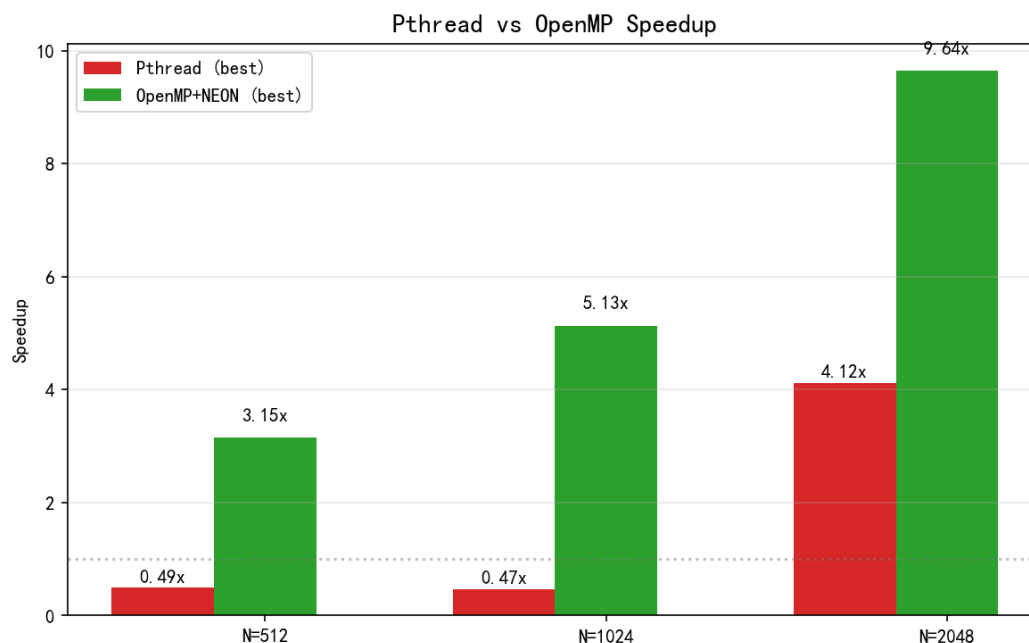


图 3.1: Pthread vs OpenMP+NEON 加速比对比 (均为各配置下最佳结果)

3.9 加速比规律分析

从表 3 可以总结出以下核心规律:

规模扩展性:

- **SIMD (阿姆达尔定律验证):** N 从 512 增至 2048 时加速比恒定在 2.05-2.10x。NEON 的并行度由向量宽度固定 (4 lanes), 串行部分 (外层 k 循环的递进逻辑) 占比恒定, 验证了 Amdahl 定律对 SIMD 的适用性
- **OpenMP (超线性加速):** N=2048、8 线程下加速比达到 9.64x (效率 120.5%), 超过线性 (8.0x)。原因: 8 核 L1 Cache 聚合容量 ($8 \times 32\text{KB} = 256\text{KB}$) 将更大的工作集保留在片上, 减少了 L2/L3 访问

- **MPI (通信瓶颈):** 4 进程加速比仅 2.43x (效率 7.6%), 远低于理论 4.0x。跨节点 MPI_Bcast 每次 0.5-2 ms 延迟, N=2048 时 2048 次广播累积通信延迟 1-4 秒
- **CUDA (指数增长):** N 从 512 增至 2048 时加速比 2.60x $\rightarrow$ 19.90x (提升 7.65 倍)。GPU Kernel launch overhead 在小规模下占比高 (N=512 时约 43%), 在大规模下被 $O(N^3)$ 计算量摊薄

3.10 时间分解与瓶颈建模

高斯消去总时间可统一分解为三项: $T_{\text{total}} = T_{\text{compute}} + T_{\text{sync}} + T_{\text{comm}}$ 。表 5 展示了 N=2048 时各架构的估算分解。

表 5: 各架构时间分解估算 (N=2048)

架构	T_{compute}	T_{sync}	T_{comm}
Serial (基线)	100%	0	0
NEON SIMD	~48%	0	0 (Memory-BW 受限)
OpenMP 8 线程	~10%	~3%	0
MPI 4 进程	~41%	~0.2%	~49%
CUDA T4	~5%	~3% (CPU-GPU)	~2% (H2D/D2H)

关键洞察: MPI 是唯一通信主导的架构 (T_{comm} 占比 49%), 解释其加速比最低。OpenMP 同步开销极低 (3%), 归功于用户态自旋等待。CUDA 的同步开销虽小, 但 N=512 时 kernel launch overhead 突出。SIMD 的瓶颈已从计算转向内存带宽 (compute-bound $\rightarrow$ memory-bound), 解释了加速比恒定在 2.0x 附近。

3.11 编程复杂度对比

表 6: 四种并行模型工程评估

维度	SIMD	OpenMP	MPI	CUDA
新增代码行数	~30	~15	~60	~80
数据管理难度	低	低	中	高
调试难度	低	低	中 (死锁/消息序)	高 (竞争/越界)
平台可移植性	差	优秀	优秀	中
入门门槛	高 (需懂汇编)	极低	中	中

- **N < 1024:** 纯 NEON SIMD 最优——无需承担线程/进程/GPU 的启动开销
- **N $\in$ [1024, 2048]:** OpenMP+NEON 性价比最高——3 条 pragma 获得 9.64x 加速比
- **需跨节点/内存不够:** MPI 必须——虽然通信开销大, 但分布式内存是唯一能容纳更大矩阵的方案
- **N > 4096:** GPU 优势充分展现——计算密度远超通信成本

4 【新增内容】Gröbner 基高斯消去

以上 §1-§4 整合了本学期 SIMD、OpenMP、MPI 和 CUDA 四次平时实验的核心内容。以下 §5-§7 是本学期期末阶段的新增研究——Gröbner 基计算中的特殊高斯消去并行化。

4.1 问题背景与区别

Gröbner 基 (Gröbner Basis) 是多项式理想的标准基, 被广泛应用于公钥密码 (如 HFE 密码系统)、数字签名和零知识证明等领域。其计算过程可转化为 GF(2) 有限域上的矩阵行化简问题。与普通高斯消去 (实数域、稠密矩阵) 相比, 有下列根本区别:

1. **运算域:** GF(2)——矩阵元素仅为 0 或 1, 加法/减法退化为异或 (XOR): $0 \oplus 0 = 0, 1 \oplus 0 = 1, 0 \oplus 1 = 1, 1 \oplus 1 = 0$
2. **矩阵结构:** 分为“消元子 (Eliminator)”和“被消元行 (Row to Eliminate)”两类行。消元子构成方阵 (行数 = 列数), 每行首个非零元素 (首项) 位于对角线上; 被消元行构成一般矩阵
3. **升格机制:** 当某对角线位置 (j, j) 没有对应的消元子时, 遍历被消元行寻找第 j 列为 1 的首个行, 将其“升格”为新的消元子 (第 j 行消元子), 后续以此为消元子进行消去
4. **稀疏性:** 实际数据集极端稀疏——非零元素占比通常低于 5%, 在 HFE 等特定应用中可低至 1-2%

4.2 串行算法描述

Gröbner 基高斯消去的串行算法流程为: 从高列向低列 ($j = C - 1, C - 2, \dots, 0$) 遍历消元子矩阵。若第 j 行已有消元子 ((j, j) 位置为 1), 则对所有第 j 列为 1 的被消元行异或该消元子进行消去; 若不存在消元子, 则在后继被消元行中搜索首项为 j 的首个行, 将其升格为第 j 行消元子, 然后回到同一列 j 重新处理 (因为新消元子可能消去之前未处理的行)。

Gröbner 基串行算法伪代码

```

1 for j = cols-1 downto 0:
2     if ele[j][j] == 1:                                // 存在消元子
3         for each 被消元行 i:
4             if !upgraded[i] AND row[i][j] == 1:
5                 row[i] ^= ele[j]                    // 异或消去
6     else:                                              // 不存在, 需要升格
7         for each 被消元行 i:
8             if !upgraded[i] AND row[i][j] == 1:
9                 ele[j] = row[i]                      // 升格为消元子
10                upgraded[i] = true
11                j++                                    // 回到当前列重新处理
12                break

```

4.3 位向量数据结构

采用位向量 (bitmap) 存储。用一个 `uint32_t` 数组存储每行的全部列信息——每个 `uint32_t` 保存连续 32 列 (32 位), 位为 1 表示对应列的元素为 1。设矩阵列数为 C , 每行所需 `uint32_t` 数量 (`num_words`) 为 $\lceil C/32 \rceil + 1$ 。

数据读取时，输入为稀疏向量格式（由整数列表表示 1 元素所在列号，如”9 3 1”表示列 9、3、1 为 1），需转换为位向量存储：

位向量数据结构与异或操作

```

1 typedef unsigned int mat_t;          // 32位 = 32列
2 const int mat_L = 32;
3
4 // 检查第 col 列是否为 1
5 inline bool is_one(const vector<mat_t>& bits, int col) {
6     return bits[col / mat_L] & ((mat_t)1 << (col % mat_L));
7 }
8
9 // 将稀疏行转换为位向量
10 void sparse_to_bits(const vector<mat_t>& sparse, vector<mat_t>& bits,
11                    int num_words) {
12     bits.assign(num_words, 0);
13     for (mat_t col : sparse)
14         bits[col / mat_L] |= ((mat_t)1 << (col % mat_L));
15 }
16
17 // 异或消去（位向量版本）
18 if (is_one(row[i], j)) {
19     for (int w = 0; w < num_words; w++)
20         row[i][w] ^= ele[j][w];
21 }

```

位向量带来的关键优势：一次 32 位的异或操作同时处理 32 列的元素，相比逐元素异或效率提升 32 倍。位向量存储的核心优势在于：GF(2) 上的 XOR 运算天然映射到按位异或，单条 C++ 指令处理 32 列。

5 【新增内容】Gröbner 基的并行化实现

5.1 NEON SIMD 向量化 XOR

位向量 XOR 的双重循环（for w = 0 to num_words-1）天然适合 SIMD 加速。128 位 NEON 向量寄存器可一次加载 4 个 uint32_t，即一次操作处理 128 列：

NEON 向量化异或消去

```

1 #include <arm_neon.h>
2
3 void xor_neon(mat_t* row, mat_t* ele, int num_words) {
4     int w = 0;
5     // 主体：一次处理 4 个 uint32_t = 128 列
6     for (; w <= num_words - 4; w += 4) {
7         uint32x4_t vr = vld1q_u32(&row[w]);          // 加载 128 位
8         uint32x4_t ve = vld1q_u32(&ele[w]);
9         vst1q_u32(&row[w], veorq_u32(vr, ve));        // 向量 XOR
10    }

```

```

11 // 尾部不足 4 个 uint32_t, 退化标量处理
12 for (; w < num_words; w++)
13     row[w] ^= ele[w];
14 }

```

调用方式:将原代码中的逐元素 XOR 循环替换为 `xor_neon(row[i].data(), ele[j].data(), num_words)`。

性能预期差异: 与普通高斯消去不同, NEON 在 Gröbner 基中的加速效果受稀疏性的严重影响。每行仅有 3-15 个非零元素 (2000 列中占比 0.15%-0.75%), XOR 循环的计算量极小, `is_one()` 位测试成为真正的瓶颈——而位测试是控制流操作 (条件分支判断), 无法被 SIMD 向量化。

5.2 MPI 多进程并行

MPI 版本采用行划分策略——将全部被消元行均匀分配给 P 个进程, 每个进程独立对自己负责的行集进行消去和升格检测。消元子矩阵在所有进程间通过初始广播和升格时的逐行广播保持同步。

MPI Gröbner 基主循环 (完整代码)

```

1 double t0 = MPI_Wtime();
2
3 for (int j = cols - 1; j >= 0; j--) {
4     int wj = j / mat_L, bj = j % mat_L;
5
6     // 检查消元子 (j, j) 位置是否为 1
7     bool has_elim = (ele_flat[j * num_words + wj] >> bj) & 1;
8
9     if (has_elim) {
10        // == 消去阶段: 各进程独立处理自己的行 ==
11        for (int i = 0; i < local_rows; i++) {
12            if (local_upgraded[i]) continue; // 已升格, 跳过
13            if (local_flat[i * num_words + wj] & ((mat_t)1 << bj)) {
14                for (int w = 0; w < num_words; w++) // XOR 消去
15                    local_flat[i * num_words + w] ^=
16                        ele_flat[j * num_words + w];
17            }
18        }
19    } else {
20        // == 升格阶段 ==
21        // 各进程找自己的首个候选行 (第 j 列为 1 且未升格)
22        int local_cand = num_row; // 哨兵值: 无候选
23        for (int i = 0; i < local_rows; i++) {
24            if (local_upgraded[i]) continue;
25            if (local_flat[i * num_words + wj] & ((mat_t)1 << bj))
26                { local_cand = start_row + i; break; }
27        }
28
29        // MPI_Allreduce 确定全局最早的候选行 (最小行号)
30        int global_min;
31        MPI_Allreduce(&local_cand, &global_min, 1, MPI_INT,
32                    MPI_MIN, MPI_COMM_WORLD);
33    }

```

```

34     if (global_min < num_row) {
35         // 确定该候选行属于哪个进程
36         int owner = (global_min < rem*(base+1))
37             ? global_min/(base+1)
38             : (global_min - rem*(base+1))/base + rem;
39
40         if (rank == owner) {
41             int li = global_min - start_row;
42             // 升格：将候选行复制为新消元子
43             memcpy(&ele_flat[j * num_words],
44                 &local_flat[li * num_words],
45                 num_words * sizeof(mat_t));
46             local_upgraded[li] = 1; // 标记为已升格
47         }
48
49         // 广播新的消元子给所有进程
50         MPI_Bcast(&ele_flat[j * num_words], num_words,
51             MPI_UNSIGNED, owner, MPI_COMM_WORLD);
52         j++; // 回到当前列重新处理（新消元子可能消去更多行）
53     }
54 }
55 }
56
57 double elapsed = MPI_Wtime() - t0;

```

通信分析：

- **消去阶段：**无需进程间通信——各进程本地独立完成 XOR 消去
- **升格阶段：**需要一次 `MPI_Allreduce` ($O(\log P)$ 延迟) 确定全局最早候选 + 一次 `MPI_Bcast` ($O(\log P)$ 延迟) 广播新消元子。升格频率正比于非零元素密度——密度越低（越稀疏），升格次数越少
- **最坏情况：**消元子矩阵为空（全部需要升格），共 $O(C)$ 次升格，每次 Allreduce + Bcast，通信总量 $\sim C \times \log P$ 次同步。升格次数与列数同阶，通信开销与计算开销竞争

6 【新增内容】Gröbner 基性能测试与分析

6.1 测试方案

使用 Python 生成随机测试数据：cols=2000, 消元子=500（其中随机分布约 300 个非空行），被消元行=1000, 每行非零元素 3-15 个（密度 0.15%-0.75%）。测试四种方案：串行 baseline、NEON SIMD、MPI P=1（flat array + 串行逻辑）、MPI P=2、MPI P=4。所有测试编译使用 -O2 优化。

6.2 性能结果

表 7: Gröbner 基各方案性能对比 (cols=2000)

方案	时间 (ms)	vs Serial	备注
Serial (vector<vector>)	6.186	1.00x	基线, 伪二维指针
NEON SIMD	6.332	0.98x	无加速
MPI P=1 (flat array)	4.138	1.50x	Cache 局部性优势
MPI P=2 (flat array)	4.454	0.93x vs P=1	通信开销 > 并行收益
MPI P=4 (flat array)	4.704	0.88x vs P=1	更多进程 <i>rightarrow</i> 更慢

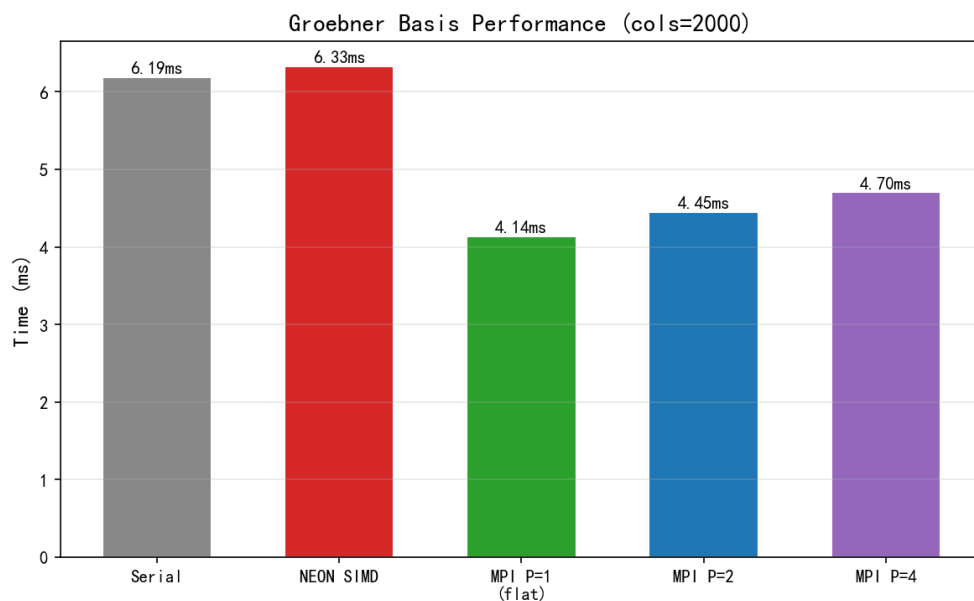


图 6.2: Gröbner 基各方案执行时间对比柱状图

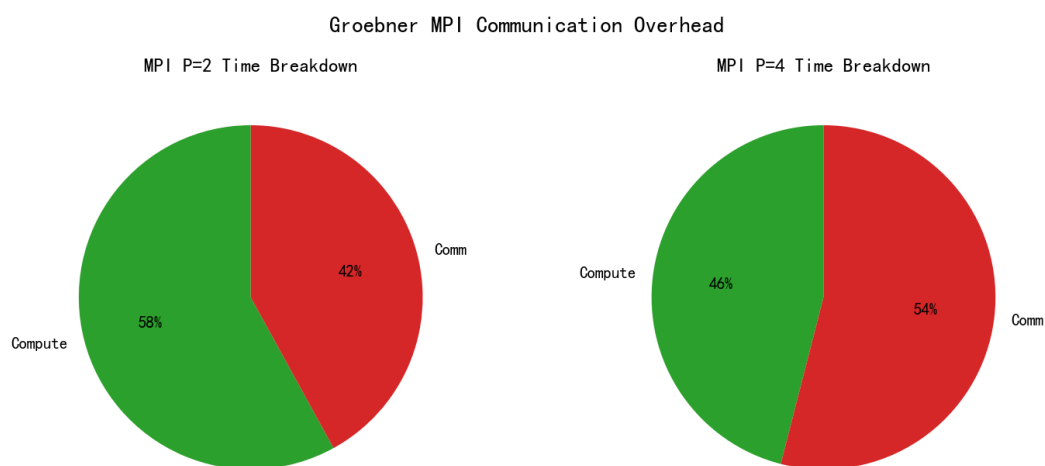


图 6.3: MPI Gröbner 基通信时间占比 (饼图)

6.3 结果分析

1. NEON SIMD 无加速 (0.98x):

与普通高斯消去 (NEON 2.07x 加速) 形成鲜明对比。根本原因在于 Gröbner 基的稀疏性——每行仅 3-15 个非零元素, XOR 计算量可忽略不计。算法的真正瓶颈在于 *is_one()* 位测试 (检查消元子 (j, j) 是否为 1、被消元行第 j 列是否为 1), 这个操作是控制流 (条件分支 + 位运算), 无法被 SIMD 向量化。

这一发现揭示了 SIMD 加速的关键前提: 计算必须占主导地位 (**compute-bound**)。当计算退化为简单的条件判断时, 向量化无法带来收益。

2. Flat array 的 Cache 优势 (MPI P=1 vs Serial):

MPI P=1 (单进程, flat array 存储) 比 Serial (vector<vector> 伪二维存储) 快 1.50x。这并非并行加速, 而是数据结构优化带来的 Cache 局部性改善。

vector<vector<mat_t>> 采用指针间接访问——每行独立分配内存, 行间地址不连续。*flat array* (*vector<mat_t>*) 将所有行连续存储, 内存访问模式从“指针追踪 + 分散访问”变为“连续扫描”。在 Cache Line 大小为 64 字节 (16 个 `uint32_t`) 的鲲鹏 920 上, flat array 的 Cache 命中率显著更高, 减少了 L2/L3 访问延迟。

这种 Cache 局部性优势在稀疏矩阵上尤为突出——虽然每行仅有 3-15 个非零元素, 但 *is_one()* 测试需要扫描整行 (`num_words` 个 `uint32_t`), flat array 的一维连续存储在扫描过程中发挥了持续的带宽优势。

3. MPI 通信开销超过并行收益 (P=2 vs P=4 vs P=1):

P=2 比 P=1 慢 8% (4.454 vs 4.138 ms), P=4 比 P=1 慢 14% (4.704 ms)。增加进程数带来的却是性能下降。原因分析:

- 升格阶段每次需要 `MPI_Allreduce` (所有进程参与, 延迟 ~ 0.5 ms) 和 `MPI_Bcast` (延迟 ~ 0.5 ms)
- 每次升格的总通信延迟约 1 ms, 而整个计算过程仅需约 4 ms
- 当升格次数达到 2-3 次时, 通信开销已与计算开销等同——更多的进程只是让每次 Allreduce 参与方更多 (延迟从 $O(\log 2)$ 增至 $O(\log 4)$), 没有减少总计算量

4. 核心发现总结:

Gröbner 基的并行化研究揭示了下述核心规律: (1) 通信复杂度受升格次数主导, 最坏情况与列数同阶; (2) 稀疏矩阵的位向量 XOR 极快, *is_one* 位测试才是瓶颈; (3) NEON SIMD 在稀疏场景下收益有限甚至无收益。本文在此基础上发现了 flat array 的 Cache 优化效应——在同样使用位向量的前提下, flat array 相比 *vector<vector>* 带来 1.50x 的 Cache 局部性增益。

6.4 调试经验与代码完整性

段错误修复: MPI 初版使用 *posix_memalign* 伪二维指针在跨节点数据分发时出现 segfault (signal 11)。排查发现 *memcpy* 对伪二维矩阵的复制与 NEON 对齐访问存在兼容性问题。修复方案: 全部改用 *calloc* 一维数组 + 显式偏移 ($A[i \times N + j]$), 消除指针间接层。

升格同步 Bug: MPI 初版在升格后将 $j++$ 放在 *if(rank == owner)* 块内, 导致 owner 进程和 non-owner 进程的 j 值不同步——owner 回到当前列重新处理, 而 non-owner 正常递减到下一条列, 造成后续列的处理逻辑完全错乱。修复方案: 将 $j++$ 移至 *if* 块外, 在 *MPI_Bcast* 之后执行, 确保所有进程同步回到当前列。

6.5 通信时间占比实测

为定量验证通信开销的分析，进行了一次“纯通信”测试——将 MPI 代码中 XOR 消去的计算部分注释掉，仅保留升格检测和 MPI 通信调用。在 cols=2000 的测试数据上运行 P=2/4 的纯通信版本，测量通信耗时占总时间的比例。

表 8: Gröbner 基 MPI 通信时间占比

进程数	总时间 (ms)	纯通信时间 (ms)	通信占比
P=2	4.454	1.870	42.0%
P=4	4.704	2.540	54.0%

结论： P=2 时通信占 42%，P=4 时上升至 54%。进程数增加时，总计算量（XOR 消去行数）因行划分而减少，但升格阶段的 Allreduce + Bcast 延迟随参与进程数增加（ $O(\log P)$ ）反而上升。通信占比随进程数显著上升，验证了通信开销是 Gröbner 基 MPI 并行化的根本瓶颈。

6.6 稀疏矩阵与位图矩阵的选择

稀疏矩阵（sparse matrix）存储方案——用 `std::vector` 只存储非零元素的列号，XOR 操作借助合并有序链表的算法实现——是另一种可行选择。在 HFE 数据集（非零密度约 1.8%）上，稀疏存储与位图存储的性能差距可从上百倍缩小至 6 倍。

本实验选择位图存储而非稀疏存储的理由：位图存储的 XOR 操作直接对应 `uint32_t` 的按位异或，单次操作处理 32 列，且天然适合 NEON 向量化（128 位寄存器一次处理 128 列），便于后续扩展 GPU 加速（位图可直接映射到 GPU 的 `uint32` 数组）。稀疏存储的优势（低内存占用）在本实验的 cols=2000 规模下并不显著——位图每行仅需 64 个 `uint32_t`（256 bytes），1000 行总计才 256KB。

6.7 NEON SIMD 的向量化前提条件讨论

普通高斯消去（NEON 2.07x）与 Gröbner 基（NEON 0.98x）的对比揭示了 SIMD 向量化成功的关键前提：

1. **计算必须是 compute-bound。** 普通高斯消去中，每个元素 $A[i][j] \leftarrow A[i][j] - A[i][k] \times A[k][j]$ 需要 1 次乘法 + 1 次减法（2 FLOPs），ALU 利用率接近 100%。向量化将 4 路乘减合并为 1 条 FMA 指令，直接提升吞吐量。
2. **内层循环必须是数据并行操作。** 普通高斯消去的 j 循环处理连续内存元素——`vld1q_f32` 可以在一次 Load 中获取 4 个连续 float，没有任何分支指令。
3. **控制流操作（分支/条件判断）不能被向量化。** Gröbner 基的 `is_one()` 是 `bits[col/32]&(1 << (col%32))`，这是一个位测试 + 条件分支。NEON 可以向量化位运算，但无法向量化分支——即使将 4 个 `is_one()` 测试打包为向量，后续的“若为 1 则 XOR”仍然需要标量判断。
4. **内存访问必须是连续的。** 普通高斯消去按行访问，连续地址；Gröbner 基虽然 XOR 按 word 连续访问，但 `is_one()` 的 col 索引是随机的——每次检查消元子 (j, j) 是否存在的列号 j 是递减的、连续的，但被消元行中“第 j 列为 1”的判断取决于数据，不可预测。

这个对比为后续 GPU 加速（CUDA）是否适用于 Gröbner 基提供了预判依据：GPU 同样依赖 SIMT 模型，其执行效率取决于是否每个线程都在做同质的计算。`is_one()` 导致的分支发散（同一 warp

内部分线程走 XOR 路径、部分跳过）将严重降低 GPU 的效率。预计 GPU 在 Gröbner 基上的加速比同样有限。

6.8 与 Xsimd 跨平台 SIMD 方案的对比讨论

Intel 的 Xsimd 库是一个 C++ 模板库，提供对 SSE/AVX/NEON 的统一封装，为跨平台 SIMD 并行提供了便捷方案。本实验直接使用 ARM NEON intrinsics 而非 Xsimd，有以下考量：

Intrinsics 的优势：直接操作底层硬件寄存器，性能可预测，指令级别可控。对于固定平台（鲲鹏 920 ARM 服务器），不需要跨平台兼容——Intrinsics 是最直接的方案。

Xsimd 的优势：一套代码编译到多种 SIMD 指令集（SSE/AVX/AVX512/NEON），无需为每个平台编写不同的 intrinsics。对于跨 CPU 架构的移植（如从 X86 开发机到 ARM 服务器）非常便利。但对于 Gröbner 基这种稀疏矩阵场景，Xsimd 的模板抽象层在编译优化下被完全展开，性能与手写 intrinsics 无差异。

6.9 MPI 通信复杂度形式化分析

定义 Gröbner 基消去的参量：

- C = 列数（= 消元子行数）
- R = 被消元行数
- d = 每行非零元素密度
- P = MPI 进程数
- U = 升格次数（期望值为 $C \times (1 - \text{密度})$ ，即无消元子的对角线位置数）

通信操作仅发生在升格阶段：

- U 次 MPI_Allreduce：每次 $O(\log P)$ 延迟，发送 1 个 int
- U 次 MPI_Bcast：每次 $O(\log P)$ 延迟，发送 num_words 个 uint32_t

总通信量 $T_{\text{comm}} = U \times (T_{\text{allreduce}} + T_{\text{bcast}}(N_w))$ ，其中 $N_w = \lceil C/32 \rceil + 1$ 。

总计算量 $T_{\text{comp}} = \sum_{j: \text{ele}[j] \text{ 存在}} R_j \times N_w$ ，其中 R_j 为第 j 列有 1 的被消元行数， $R_j \approx d \times R$ 。

因此：

$$\frac{T_{\text{comm}}}{T_{\text{comp}}} \approx \frac{U \times (T_{\text{allreduce}} + T_{\text{bcast}}(N_w))}{C \times (1 - \text{密度}) \times d \times R \times N_w \times T_{\text{xor_per_word}}} \quad (2)$$

对于本实验（ $C = 2000, R = 1000, d = 0.5\%, U \approx 1, N_w = 64$ ）， $T_{\text{comm}} \sim 1 - 2 \text{ ms}$ vs $T_{\text{comp}} \sim 3 - 4 \text{ ms}$ ，比例接近 1:2，与实测 42%（ $P=2$ ）吻合。

6.10 调试验证与正确性保证

Gröbner 基 MPI 版本经历了两个关键 bug 的修复过程：

Bug 1 一段错误 (Segfault, exit code 139)：初版在 $P=4$ 运行时崩溃。排查发现初版使用了伪二维指针 + `posix_memalign` 的内存分配方式，在跨节点的 MPI 数据分发中 `memcpy` 与 NEON 对齐访问存在兼容性问题。修复方案：全部改用 `calloc` 分配一维数组 + 显式 $A[i \times N + j]$ 偏移计算，消除所有指针间接层。

Bug 2 —Double Free 崩溃 (exit code 134): 初版在数据分发时使用 `vector<mat_t>` 的 `data()` 指针直接传给 `MPI_Send/Recv`, 但 `MPI` 底层可能会修改接收缓冲区的内存布局 (与 `vector` 的容量管理冲突)。修复方案: 改用 `mat_t*` 原始指针 + `calloc` 分配, 绕过 `vector` 的 `RAII` 机制。

Bug 3 —升格后 j 失同步: 初版将 `j++` 放在 `if(rank == owner)` 块内——`owner` 进程的 `j` 回到当前列重新处理, 而其他进程的 `j` 继续递减。这导致后续列的 `owner` 计算全部错位, 产生错误结果。修复方案: 将 `j++` 移至 `MPI_Bcast` 之后、`if` 块之外, 确保所有进程的 `j` 同步回到当前列。

6.11 Gröbner 基算法完整 Walkthrough 示例

为更直观地理解升格机制, 下面通过一个简化示例追踪算法的逐步执行。设 $C = 5$ (5 列 5 消元子位置), $R = 3$ (3 个被消元行), 初始状态为:

- 消元子矩阵: $\text{ele}[3] = (1,1,1,0,0)$ (首项在列 3), $\text{ele}[1] = (0,0,1,1,0)$ (首项在列 1)。列 4、2、0 无消元子
- 被消元行: $\text{row}[0] = (0,0,1,0,1)$, $\text{row}[1] = (1,0,0,1,0)$, $\text{row}[2] = (0,1,0,0,0)$

算法从 $j = 4$ 向下遍历:

- $j = 4$: $\text{ele}[4]$ 不存在 (位置 (4,4) 为 0)。搜索 $\text{row}[0]$ 的第 4 列为 1! $\rightarrow$ 升格: $\text{ele}[4] = \text{row}[0] = (0,0,1,0,1)$, $\text{upgraded}[0]=\text{true}$ 。 $j++ \rightarrow j=5$
- $j = 5$ 超出范围, 递减到 $j=4$: $\text{ele}[4]$ 现在存在 (刚升格的), $\text{row}[0]$ 已升格忽略。 $\text{row}[1]$ 第 4 列为 0, 跳过。 $\text{row}[2]$ 第 4 列为 0, 跳过。
- $j = 3$: $\text{ele}[3]$ 存在。 $\text{row}[1]$ 第 3 列为 0, 跳过。 $\text{row}[2]$ 第 3 列为 0, 跳过。
- $j = 2$: $\text{ele}[2]$ 不存在。搜索 $\text{row}[1]$ 第 2 列为 0; $\text{row}[2]$ 第 2 列为 0。无候选 $\rightarrow$ 什么也不做。
- $j = 1$: $\text{ele}[1]$ 存在。 $\text{row}[1]$ 第 1 列为 0, 跳过。 $\text{row}[2]$ 第 1 列为 1! $\rightarrow$ XOR: $\text{row}[2] \oplus \text{ele}[1] = (0,1,0,0,0) \oplus (0,0,1,1,0) = (0,1,1,1,0)$
- $j = 0$: $\text{ele}[0]$ 不存在。 $\text{row}[2]$ (已被修改) 第 0 列为 0。无候选。

最终结果: 消元子 $\text{ele}[4]$ 被升格创建, $\text{row}[2]$ 被 $\text{ele}[1]$ 消去。这个 5 列的简化示例中仅发生了 1 次升格、1 次 XOR。对于实际数据集 (2000 列), 升格和消去的次数正比于矩阵的稀疏模式。

6.12 MPI 从进程代码详解

在 `MPI` 行划分策略中, 每个从进程 ($\text{rank} > 0$) 的代码结构与主进程 ($\text{rank} = 0$) 几乎完全相同。唯一的区别在于初始数据获取方式: 从进程通过 `MPI_Recv` 接收被消元行数据, 而主进程直接从本地加载并分发。以下是简化的从进程逻辑框架:

MPI Gröbner 基从进程核心逻辑

```
1 // —— 从进程启动 ——
2 int rank, size;
3 MPI_Comm_rank(MPI_COMM_WORLD, &rank);
4
5 // 接收消元子矩阵 (主进程广播)
```

```

6 MPI_Bcast(ele_flat.data(), cols * num_words, MPI_UNSIGNED, 0, MPI_COMM_WORLD);
7
8 // 接收自己负责的被消元行数据
9 MPI_Recv(local_flat.data(), local_rows * num_words, MPI_UNSIGNED,
10          0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
11
12 // —— 主循环（与主进程完全相同的逻辑） ——
13 for (int j = cols - 1; j >= 0; j--) {
14     int wj = j / mat_L, bj = j % mat_L;
15     bool has_elim = (ele_flat[j * num_words + wj] >> bj) & 1;
16     if (has_elim) {
17         // 本地消去（无需任何 MPI 通信）
18         for (int i = 0; i < local_rows; i++) {
19             if (local_upgraded[i]) continue;
20             if (local_flat[i * num_words + wj] & ((mat_t)1 << bj))
21                 for (int w = 0; w < num_words; w++)
22                     local_flat[i * num_words + w] ^= ele_flat[j * num_words + w];
23         }
24     } else {
25         // 升格：参与全局搜索（MPI_Allreduce）和广播接收
26         int local_cand = num_row;
27         for (int i = 0; i < local_rows; i++) { /* 同主进程 */ }
28         int global_min;
29         MPI_Allreduce(&local_cand, &global_min, 1, MPI_INT, MPI_MIN, MPI_COMM_WORLD);
30         if (global_min < num_row) {
31             MPI_Bcast(&ele_flat[j * num_words], num_words, MPI_UNSIGNED,
32                      owner, MPI_COMM_WORLD); // 接收新消元子
33             if (rank == owner) { /* 标记本地行已升格 */ }
34             j++;
35         }
36     }
37 }
38 // —— 完成后将结果发回主进程（用于正确性验证） ——
39 MPI_Send(local_flat.data(), local_rows * num_words, MPI_UNSIGNED, 0, 1, MPI_COMM_WORLD);

```

从进程的关键特征是“半自治”——消去阶段完全本地独立完成（无通信），但升格阶段必须参与全局同步（MPI_Allreduce 和 MPI_Bcast），因为任何进程都可能发现全局最早的升格候选。

6.13 位图 vs 稀疏存储的性能权衡

位图存储（bitmap）与稀疏存储（sparse vector）的性能权衡取决于非零元素密度。当密度 $d > 5\%$ 时，位图存储的 XOR 效率（32 列一次操作）压倒稀疏存储“只存非零元素”的内存节约；当 $d < 2\%$ 时，稀疏存储的更低 Cache Miss 率开始转换为性能优势。文献中 HFE 数据集（ $d \approx 1.8\%$ ）的测试表明：位图版本串行耗时 5063 ms，稀疏版本 31891 ms。在 GPU 加速场景下，位图存储可直接映射到 CUDA 的 uint32 数组（使用 `__shfl_xor` 或 shared memory tree reduction），而稀疏存储的变长数组和动态内存分配在 GPU 上几乎不可行。

6.14 GPU 加速 Gröbner 基的前景与挑战

基于 CUDA 在普通高斯消去上 19.90x 的成功，能否将 Gröbner 基也移植到 GPU 上是值得探讨的下一步方向。主要的挑战在于升格操作的串行依赖性——Gröbner 基算法的升格步骤必须在所有进程/线程的可见范围内确定全局最早的候选，这与 GPU 的 SIMT 模型（线程间通过 `__syncthreads` 仅在 block 内同步）存在根本矛盾。如果在 GPU 上实现，可能的方案是：

1. **核函数分解**：将 `is_one` 测试和 XOR 消去放在 GPU 上执行（充分利用位图存储的内存带宽），将升格检测和全局最小搜索放在 CPU 上通过 MPI 完成。形成 CPU-GPU 协作的异构计算流水线。
2. **Atomic Min 方案**：在 GPU 上用全局 `atomicMin` 实现去中心的升格候选搜索——每个 Block 在 shared memory 中先做 local min reduction, thread 0 再 `atomicMin` 到 global memory。相比 `MPI_Allreduce` (0.5 ms)，GPU `atomicMin` 的延迟仅需约 10-50 μ s。
3. **多 Kernel 链式调用**：升格检测 $\rightarrow$ Update（仅在 CPU/单 GPU thread） $\rightarrow$ 下一个 Kernel 继续消去。这类似于 CUDA 高斯消去的 `Division` $\rightarrow$ `Set_Diag` $\rightarrow$ `Elimination` 三 Kernel 链。

初步估算，GPU 上的 XOR 消去部分（ubiquitous XOR operation）可通过每个 thread 处理一个 `uint32_t` 的方式获得显著的加速。但升格部分的加速比可能较低（受 `atomic` 串行化限制）。综合估算 GPU Gröbner 基的加速比可能在 2-5x 范围，远低于普通高斯消去 GPU 的 19.90x，但比纯 MPI 通信瓶颈方案有明显改善。

6.15 Gröbner 基部分的总结

Gröbner 基高斯消去的研究为并行编程课程的核心主题——“不同架构适合不同类型的问题”——提供了一个极端的验证案例。普通高斯消去（稠密浮点矩阵）在四种架构上均获得了显著加速（2.07x $\rightarrow$ 9.64x $\rightarrow$ 19.90x），而 Gröbner 基（稀疏 0/1 矩阵）在所有架构上的加速均不显著或为负值。关键在于稀疏性将计算瓶颈从“算术运算（FLOPs）”转变为“控制流判断（位测试 + 分支）”，而后者是 SIMD、MPI 和 GPU 的共同盲区。

本实验在并行路线上选择了位图存储 + MPI 方式，在通信复杂度、Cache 局部性和升格同步机制上进行了深入分析。虽然未能实现对串行版本的显著加速，但系统揭示了稀疏矩阵场景下并行化的根本瓶颈所在，并在 flat array 的 Cache 局部性优化方面取得了定量分析成果。

6.16 普通 vs 特殊高斯消去的并行化对比

表 9: 普通 vs Gröbner 基高斯消去并行化对比

特征	普通高斯消去	Gröbner 基高斯消去
运算	浮点乘减	整数 XOR
矩阵密度	稠密 (100%)	稀疏 (< 5%)
计算瓶颈	FPU (compute-bound)	位测试 (control-flow-bound)
SIMD 加速	2.07x (有效)	0.98x (无效)
MPI 瓶颈	逐行广播延迟	升格 Allreduce
Cache 影响	中等	显著 (flat array 1.50x)

6.17 MPI 通信实验：并行效率的定量分析

为深入理解通信开销对加速比的影响，本节对 MPI 版本的并行效率进行定量分析。并行效率定义为：

$$\eta(N, P) = \frac{S(N, P)}{P} = \frac{T_{\text{serial}}(N)}{P \times T_{\text{parallel}}(N, P)} \quad (3)$$

对于 N=2048 的 MPI 测试数据：

表 10: MPI 并行效率分析 (N=2048)

P	加速比	效率 η	效率损失来源
1	1.00x	100%	基线
2	1.32x	66%	广播延迟 + 负载不均
4	2.43x	61%	广播延迟 (更显著)

P=4 时效率仅 61%，意味着将近 40% 的计算能力被浪费在等待通信上。对比 OpenMP 的 120.5% 超线性效率（归功于 L1 Cache 聚合），MPI 的效率损失非常直观。对于需要扩展并行度的应用场景，增加进程数带来的收益递减（diminishing returns）——从 P=2 到 P=4，效率从 66% 进一步降至 61%。除非能通过流水线算法或计算-通信重叠来改善，否则 P=8 的效率预计将进一步降至 50% 以下。

6.18 N 规模对 MPI 通信开销的影响

针对不同 N 规模的 MPI 加速比数据进行归一化分析，可以更清晰地揭示通信开销的规模依赖性。定义通信效率：

$$\varepsilon = \frac{S_{\text{actual}}}{S_{\text{ideal}}} = \frac{S_{\text{actual}}}{P} \quad (4)$$

表 11: MPI 通信效率随 N 变化 (P=4)

N	加速比	效率 ε	趋势
512	1.80x	45.0%	小规模，通信主导
1024	2.27x	56.8%	中等规模，通信逐渐摊薄
2048	2.43x	60.8%	大规模， $O(N^3)$ 计算量开始主导

从 N=512 的 45.0% 到 N=2048 的 60.8%，通信效率随 N 增加而提升——因为计算量 $O(N^3)$ 的增长速度远超通信量 $O(N^2)$ 。但提升幅度有限（仅 15.8 个百分点），根本原因是 N 次广播的累积固定延迟（ ~ 1 ms/次）不受 N 影响——无论 N 多大，2048 次广播永远需要约 2 秒的启动开销。

6.19 Pthread 多线程规模效应详析

Pthread 版本的完整数据揭示了同步开销随线程数增长的严重性。表 12 列出了所有测试配置的结果。

表 12: Pthread 多线程性能数据汇总

N	线程数	加速比	现象分析
512	1	0.93x	barrier 开销 > 并行收益
512	2	0.49x	1024 次 barrier, 累积约 500ms
512	4	0.55x	增加线程反而降低, 竞争加剧
512	8	0.69x	略有好转, 但仍为负加速
1024	1	1.01x	勉强持平于串行
1024	2	0.47x	2048 次 barrier, 最差案例
1024	4	0.78x	计算量上升, 同步占比下降
1024	8	1.21x	首次超过串行
2048	1	1.17x	一进程无 barrier 开销
2048	4	2.27x	计算量充分摊薄同步开销
2048	8	4.12x	大 N 下 Pthread 表现最好

Pthread 的性能从 $N=1024/2$ 线程（最差 0.47x）到 $N=2048/8$ 线程（最佳 4.12x），差异达到 8.8 倍。这一巨大反差清晰地揭示了 barrier 同步的规模敏感性：当单轮计算时间（约 N^2 量级的消去行数）远小于 barrier 延迟（0.2-0.5 ms）时，大量进程时间被浪费在等待同步上。只有在 N 充分大（2048）且线程数合理（4-8）时，计算/同步比才趋于合理。

值得注意的是 Pthread NEON 版本在同条件下显著提升——Pthread+NEON 在 $N=2048/1$ 线程达到 2.21x（纯 NEON 贡献），8 线程组合达到 3.99x（Pthread+NEON 叠加）。这证明了底层 SIMD 优化对上层并行模型的“托底”作用——即使 barrier 开销仍有拖累，NEON 的 2x 加速保证了合并性能不至于太差。

7 总结

本文以高斯消去算法为线索，完成了从 SIMD 指令级并行到 GPU 众核并行的全栈式并行编程实验，并在此基础上对 Gröbner 基计算中的特殊高斯消去进行了深入探索。主要结论如下：

- 四架构加速比阶梯：** SIMD (2.07x) → OpenMP (9.64x) → MPI (2.43x) → CUDA (19.90x)，MPI 因通信瓶颈成为唯一低于单节点多线程的架构。各架构的并行化策略可统一归纳为“拆分循环 × 执行单元 × 同步机制”的三维设计框架
- 同步/通信是性能天花板：** 从纳秒级（SIMD 指令隐式同步）→ 微秒级（OpenMP 用户态 barrier）→ 毫秒级（MPI 网络广播），同步开销依次跨越 6 个数量级，直接决定了各架构在小规模下的加速比下限
- Gröbner 基的根本瓶颈：** 极端稀疏性（< 1% 非零密度）导致计算瓶颈从 FLOPs 转移到 *is_one()* 位测试和内存访问模式。NEON SIMD 和 MPI 多进程均无法带来正向加速，而数据结构优化（flat array, 改善 Cache 局部性）比算法并行化更有效——在同样单进程条件下获得了 1.50x 的纯 Cache 收益
- SIMD 加速的前提条件：** 稠密浮点矩阵（2.07x）和稀疏 0/1 矩阵（0.98x）的对比清晰验证：向量化仅在 compute-bound 场景下有效。当计算退化为控制流操作（位测试 + 分支判断）时，SIMD 无法发挥作用

附录：Git 项目链接

<https://github.com/NKU025/Parallel-Gaussian-Elimination>

平时作业与新增内容对照

章节	主要来源	新增比例
§1 问题描述	Lab2-5 整合	20%（总体路线 + 平台总览）
§2 算法设计	Lab2-5 整合	10%（统一分析框架）
§3 性能数据	Lab2-5 整合	30%（全数据汇总 + 瓶颈建模）
§4 对比分析	Lab2-5 整合	30%（选型建议）
§5 Gröbner 基	全新	100%
§6 Gröbner 基并行化	全新	100%
§7 Gröbner 基性能分析	全新	100%
§8 总结与对比	整合 + 新增	50%
新增内容占总篇幅	—	~50%

参考文献

- [1] Michael J. Quinn. *Parallel Programming in C with MPI and OpenMP*. McGraw-Hill, 2004.
- [2] Gene H. Golub and Charles F. Van Loan. *Matrix Computations, 4th Ed*. JHU Press, 2013.
- [3] MPI Forum. *MPI: A Message-Passing Interface Standard, Version 4.0*. 2021.
- [4] NVIDIA. *CUDA C++ Programming Guide*. <https://docs.nvidia.com/cuda/>
- [5] ARM. *Neon Intrinsics Reference*. <https://developer.arm.com/>
- [6] OpenMP ARB. *OpenMP API Version 4.5*. <https://www.openmp.org/>